

Minimum Number of Semesters

There are n courses.

Each relation

$$u \rightarrow v$$

means

You must finish course u before taking course v .

Unlike Course Schedule,
you can study multiple courses simultaneously.

The question is:

What is the minimum number of semesters required to complete all courses?

If impossible,
return -1.

Example 1

$$n = 3$$

$$1 \rightarrow 3$$

$$2 \rightarrow 3$$

Graph



Semester 1

1 2

Both have no prerequisites.

Since parallel study is allowed,
take both together.

Semester 2

3

Answer

2

Example 2

1 → 2

2 → 3

3 → 4

Graph

```
1 -> 2 -> 3 -> 4
```

Semester

1 2 3 4

Answer

4

Example 3

1 → 2

2 → 3

3 → 1

Cycle.

Impossible.

Return

-1

Question

How many layers does Kahn have?

Algorithm

Exactly same Kahn.

Only addition:

Before processing one level,
store

```
int size = q.size();
```

Now

```
while(size-- > 0) {  
    process one course  
}
```

After finishing this level,

```
semester++;
```

Code Implementation

```
class Solution {  
public:  
    int minimumSemesters(int n, vector<vector<int>>& relations) {  
  
        vector<vector<int>> adj(n + 1);  
        vector<int> indegree(n + 1, 0);  
  
        for(auto &it : relations) {  
            int u = it[0];  
            int v = it[1];  
            adj[u].push_back(v);  
            indegree[v]++;  
        }  
  
        queue<int> q;  
        for(int i = 1; i <= n; i++) {  
            if(indegree[i] == 0) {  
                q.push(i);  
            }  
        }  
    }  
};
```

```

    }
}

int semester = 0;
int coursesCompleted = 0;

while(!q.empty()) {

    // all courses currently in the queue
    // can be taken in the same semester
    int size = q.size();
    semester++;

    while(size-->0) {

        int curr = q.front();
        q.pop();
        coursesCompleted++;

        for(int neighbour : adj[curr]) {

            indegree[neighbour]--;
            if(indegree[neighbour] == 0) {
                q.push(neighbour);
            }
        }
    }
}

if(coursesCompleted == n) {
    return semester;
}
//otherwise cycle exists
return -1;
}
};

```